

Unit 1 Software Process and Model

Introduction to Software Engineering: Objectives,
Principles, and Practices



Agenda

- Definition of Software Engineering
- Objectives of Software Engineering
- Key Principles of Software Engineering
- Common Practices in Software Engineering

What is Software Engineering?

•Definition:

Software Engineering is the application of engineering principles to the development of software in a systematic method.

•Importance:

Ensures reliability, efficiency, and maintainability of software systems.

Objectives of Software Engineering

- **Objective 1:** Deliver high-quality software that meets customer needs.
- **Objective 2:** Ensure the software is reliable and performs efficiently.
- **Objective 3:** Make software maintainable and scalable.
- **Objective 4:** Optimize cost and time of software development.
- **Objective 5:** Facilitate team collaboration and communication.

- **Software:** It is a collection of integrated programs. Means it carefully-organized instructions and code written by developers on any of various particular computer languages.
- **Engineering:** It is the application of scientific and practical knowledge to invent, design, build, maintain and improve frameworks, processes, etc.
- **Software Engineering:** It is an engineering branch related to the evolution of software product using well-defined scientific principles, techniques and procedures.

The result of software engineering is an effective and reliable software product.

Need of Software Engineering

1. **Handling Big Projects:** Corporation must use SE to handle large projects without any issues.
2. **To manage the cost:** Software engineering programmers plan everything and reduce all those things that are not required.
3. **To decrease time:** It will save a lot of time if you are developing software using a software engineering technique.
4. **Reliable software:** It is the company's responsibility to deliver software products on schedule and to address any defects that may exist.
5. **Effectiveness:** Effectiveness results from things being created in accordance with the software standards.

Key Principles of Software Engineering

- **Principle 1: Abstraction** : Simplifying complex systems by modeling classes appropriate to the problem.
Benefits: Easier to understand, manage, and modify.
- **Principle 2: Modularity** : Dividing a software system into discrete modules that can be developed independently.
Benefits: Enhances reusability, maintainability, and parallel development.
- **Principle 3: Encapsulation** : Bundling the data with the methods that operate on the data.
Benefits: Protects the internal state of an object and promotes modularity.
- **Principle 4: Separation of Concerns**: Dividing a program into distinct sections, where each section addresses a separate concern.
Benefits: Reduces complexity and enhances readability and maintainability.

-
- **Principle 5: Incremental Development** : Developing software in small, manageable increments.

Benefits: Allows for early detection of issues, user feedback, and continuous improvement.

- **Principle 6: Maintainability** : Writing code in a way that it can be easily understood, modified, and extended by others.

Benefits: Reduces future costs and effort in maintaining the software.

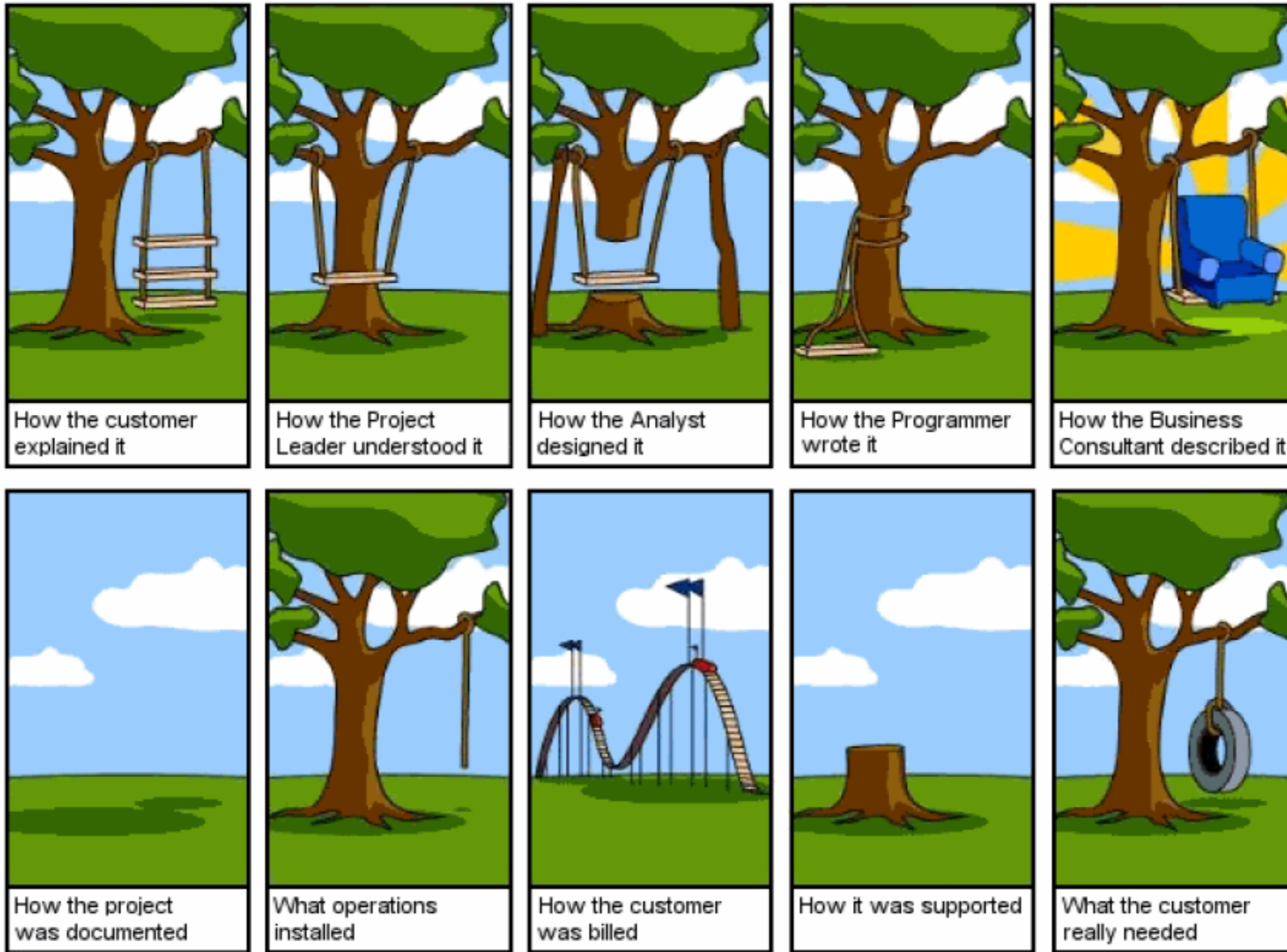
- **Principle 7: Scalability**: Designing software to handle growth, whether in data volume, number of users, or transaction frequency.

Benefits: Ensures long-term usability and performance.

Common Practices in Software Engineering

- **Practice 1: Requirement Analysis** : Gathering and defining what the software needs to do.
Tools: User stories, use cases, interviews.
- **Practice 2: System Design**: Planning the architecture and components of the system.
Tools: UML diagrams, design patterns, architectural blueprints.
- **Practice 3: Implementation (Coding)**: Writing the actual code according to the design specifications.
Tools: IDEs, version control systems, coding standards.
- **Practice 4: Testing** : Verifying that the software works as intended and is free of defects.
Types: Unit testing, integration testing, system testing, acceptance testing.
- **Practice 5: Deployment** : Delivering the software to the end-users.
Methods: Continuous integration/continuous deployment (CI/CD), manual deployment.
- **Practice 6: Maintenance** : Updating and improving the software post-deployment.
Activities: Bug fixing, adding new features, performance optimization.

Reality in Software Development



Characteristics of Good Software

Operational	Transitional	Maintenance
Budget	Interoperability	Flexibility
Efficiency	Reusability	Maintainability
Usability	Portability	Modularity
Dependability	Adaptability	Scalability
Correctness		
Functionality		
Safety		
Security		

Nature of Software

1. System Software:

- System software is software designed to provide a platform for other software's.
- It is a interaction between hardware & application software.
- **Examples :** Operating Systems like macOS, Linux, Android and Microsoft Windows.

2. Application Software:

- Application Software is a computer program designed to carry out a specific task as per user & business need.
- **Examples:** Social medias apps, Gaming apps, Word processing apps, Multimedia apps, Banking apps, Shopping apps, Booking apps etc.



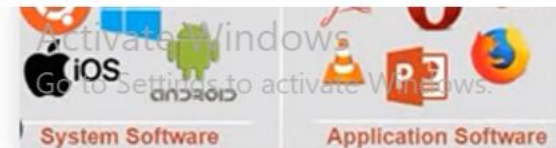
Nature of Software

3. Engineering and Scientific Software:

- This software is used to facilitate the engineering function and task take real time.
- It has very high accuracy, complex formula evolution & data analysis.
- **Examples:** Weather prediction apps, Stock Market apps, Stress Analysis, Body measurement apps

4. Embedded Software:

- Embedded software resides within the system or product and is used to implement and control feature and function for the end-user and for the system itself.
- **Example:** Switches, Routers, Digital camera, Washing machine functionalities, Traffic control etc.



Nature of Software

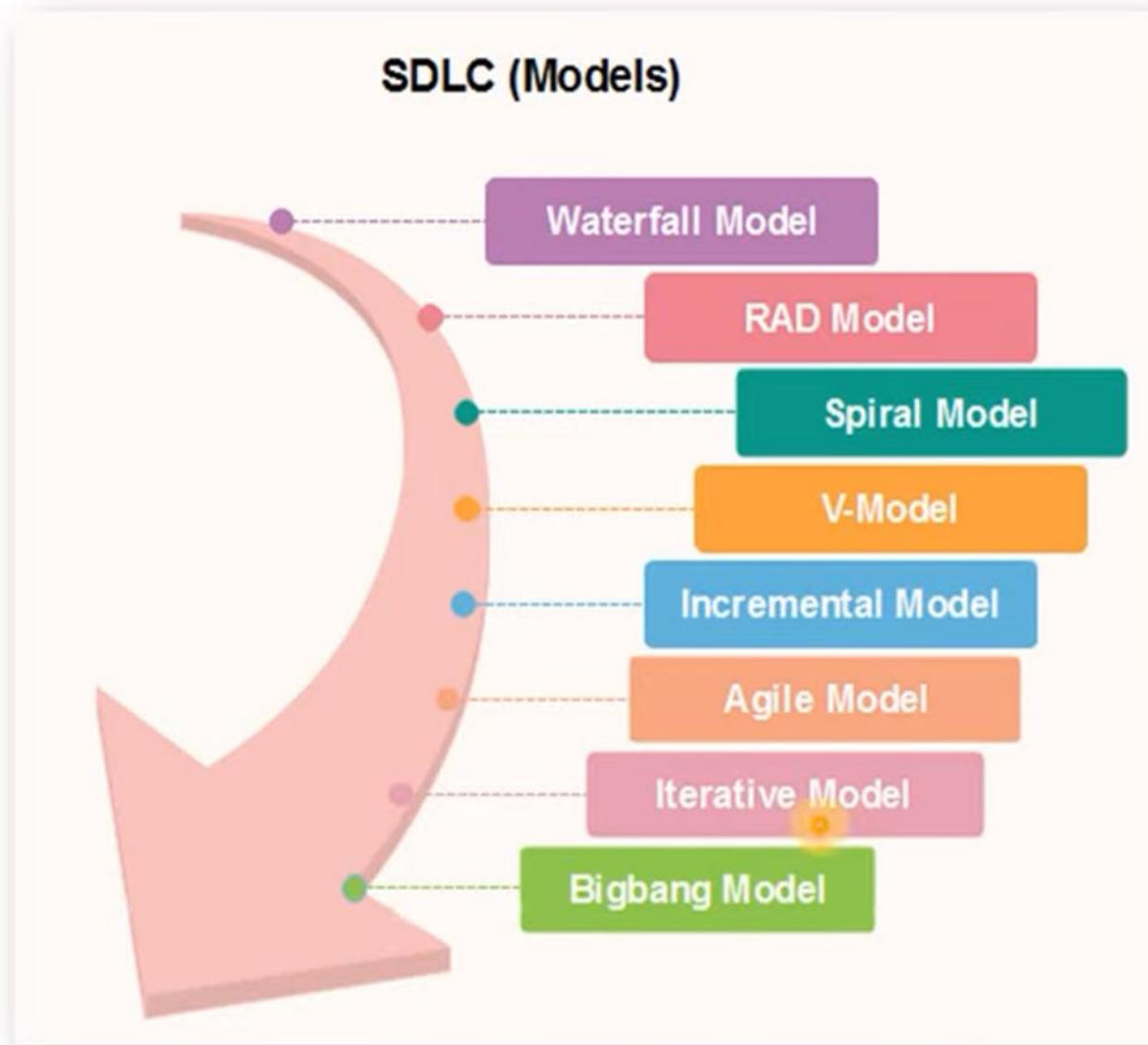
5. Web Applications:

- It is a client-server computer program which the client runs on the web browser.
- Web apps can be little more than a set of linked hypertext files that present information using text and limited graphics.
- **Examples:** Online forms, Shopping carts, Gmail, Yahoo, Photo editing, File conversion etc.

6. Artificial Intelligence Software:

- It makes use of a nonnumerical algorithm to solve a complex problem.
- Application within this area includes robotics, expert system, pattern recognition, artificial neural network, theorem proving and game playing.
- **Examples:** Google Cloud, Azure studio, Tensor Flow, Salesforce etc.

Software Engineering – SDLC Models



Activate
Go to Setting



Attributes Of Good Software

- Maintainability
 - Software must evolve to meet changing needs.
- Dependability
 - Software must be trustworthy.
- Efficiency
 - Software should not make wasteful use of system resources.
- Usability
 - Software must be usable by the users for which it was designed.

Software Myths

- Common Software Myths
 - Management Myths
 - Customer Myths
 - Developer Myths
- Impact of Software Myths
- Debunking the Myths

Management Myths

- Myth 1: "If we get behind schedule, we can add more programmers to catch up."

Reality: Adding more programmers to a late project often makes it later due to the ramp-up time and increased communication overhead (Brooks' Law).

Impact: Can lead to increased complexity, miscommunication, and reduced productivity.

- Myth 2: "Project requirements are clear from the beginning and do not change."

Reality: Requirements often evolve as stakeholders better understand their needs and the possibilities.

Impact: Can result in scope creep, requiring flexible and iterative development processes.



Management Myths

- Myth 3: "Software development is a straightforward process similar to manufacturing."

Reality: Software development is a creative process involving discovery and problem-solving, unlike predictable manufacturing processes.

Impact: Misalignment in expectations, leading to frustration and inefficiencies.

Customer Myths:

Myth 1: "A general statement of objectives is sufficient to start coding."

Reality: Clear, detailed requirements are crucial for successful development.

Impact: Vague objectives can lead to misinterpretation, rework, and project failure.

-
- Myth 2: "Software can be changed easily at any time."

Reality: Changes, especially late in the development process, can be costly and time-consuming.

Impact: May lead to significant redesign, regression bugs, and schedule delays.

- Myth 3: "The final product will work perfectly as initially envisioned."

Reality: The final product often requires iterations and refinements based on testing and user feedback.

Impact: Unrealistic expectations can result in dissatisfaction and perceived project failure.

Developer Myths

- **Myth 1: "Once the program is written and works, our job is done."**

Reality: Maintenance, support, and updates are ongoing responsibilities.

Impact: Neglecting post-release phases can lead to unaddressed bugs and technical debt.

- **Myth 2: "The only deliverable is the working program."**

Reality: Documentation, testing artifacts, and user manuals are also critical deliverables.

Impact: Lack of comprehensive deliverables can hinder future maintenance and user adoption.

- **Myth 3: "We can always fix problems later."**

Reality: Deferring fixes can escalate issues, making them harder and costlier to address.

Impact: Accumulated technical debt and degraded software quality over time.

Impact of Software Myths

- **Negative Consequences:**
 - Misaligned expectations
 - Inefficiencies and delays
 - Increased costs
 - Poor software quality
 - Team frustration and burnout

Debunking the Myths

- **Strategies: Education and Training:** Equip all stakeholders with accurate knowledge.
- **Communication:** Foster open, honest dialogue about project realities.
- **Process Improvement:** Adopt agile practices, continuous integration, and iterative development.
- **Realistic Planning:** Set achievable goals and timelines, acknowledging potential changes.

Principles of Software Engineering

- The Reason It All Exists
- KISS (Keep It Simple, Stupid!)
- Maintain the vision
- What You Produce, Others Will Consume
- Be Open to the Future
- Plan Ahead for Reuse
- Think

First principle: The Reason It All Exists

- A software system exists for one reason: *to provide value to its users.*
- Before specifying a system requirement, before noting a piece of system functionality, before determining the hardware platforms or development processes, ask yourself questions such as: “Does this add real value to the system?”
- If the answer is “no,” don’t do it

The Second Principle: *KISS (Keep It Simple, Stupid!)*

- *All design should be as simple as possible, but no simpler.*
- This facilitates having a more easily understood and easily maintained system.
- It often takes a lot of thought and work over multiple iterations to simplify.
- The payoff is software that is more maintainable and less error-prone

The Third Principle: *Maintain the Vision*

- *A clear vision is essential to the success of a software project.*
- Having an **empowered architect** who can hold the vision and **enforce compliance** helps ensure a very successful software project.

The Fourth Principle: *What You Produce, Others Will Consume*

- someone else will use, maintain, document, or otherwise depend on being able to understand your system.
- *always specify, design, and implement knowing someone else will have to understand what you are doing.*

The Fifth Principle: *Be Open to the Future*

- A system with a long lifetime has more value.
- software systems must endure far longer. To do this successfully, these systems must be ready to adapt to these and other changes.

The Sixth Principle: *Plan Ahead for Reuse*

- Reuse saves time and effort
- The reuse of code and designs has been proclaimed as a major benefit of **using object-oriented technologies**
- *planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.*

The Seventh principle: *Think!*

- *Placing clear, complete thought before action almost always produces better results.*

Challenges Of Software Engineering

The key challenges facing software engineering are

- **Coping with legacy systems**, coping with increasing diversity and coping with demands for reduced delivery times.
- **Legacy systems**
 - Old, valuable systems must be maintained and updated.
- **Heterogeneity**
 - Systems are distributed and include a mix of hardware and software.
- **Delivery**
 - There is increasing pressure for faster delivery of software

Summary:

- Software myths can significantly impact project success.
- Awareness and proactive measures are key to debunking myths.
- Continuous improvement and realistic planning lead to better software outcomes.